

```

import streamlit as st

import pandas as pd

import numpy as np

import plotly.express as px

import plotly.graph_objects as go

import joblib

import time

from datetime import datetime

import whatsapp


# --- Configuration & Styling ---

st.set_page_config(page_title="Eco-Scale Dashboard", page_icon="🌿", layout="wide",
initial_sidebar_state="expanded")


# Theme Toggle in Sidebar

light_mode = st.sidebar.toggle("☀️ Light Mode Toggle", value=False, help="Switch between Dark and
Light aesthetic.")


if not light_mode:

    # Dark Mode CSS (Highly Aesthetic)

    st.markdown("""

    <style>

    /* Hide Streamlit Clutter */

    #MainMenu {visibility: hidden;}

    footer {visibility: hidden;}

    header {visibility: hidden;}

    /* App Background */

    .stApp { background-color: #0d1117; color: #c9d1d9; }

    .css-1d391kg, .css-18ni7ap { background-color: #161b22; border-right: 1px solid #30363d; }

    /* Glassmorphic Metric Cards */

```

```

.metric-card {
    background: #161b22;
    border: 1px solid rgba(255,255,255,0.05);
    border-radius: 12px;
    padding: 20px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.2);
    transition: all 0.3s cubic-bezier(0.25, 0.8, 0.25, 1);
    position: relative;
    overflow: hidden;
}

.metric-card:hover {
    transform: translateY(-4px);
    box-shadow: 0 8px 20px rgba(32, 201, 151, 0.1);
    border: 1px solid rgba(32, 201, 151, 0.3);
}

.metric-value { font-size: 38px; font-weight: bold; color: #ffffff; text-shadow: 0 0 10px
rgba(255,255,255,0.1); }

.metric-title { font-size: 13px; color: #20c997; text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }

h1, h2, h3 { color: #58a6ff; }

/* Unique Feature Box */
.unique-feature {
    background: rgba(19, 35, 58, 0.3);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(88, 166, 255, 0.2);
    border-radius: 16px; padding: 25px; margin-top: 20px;
    box-shadow: inset 0 0 20px rgba(88,166,255,0.05);
}

```

```

/* Modern Pill Tabs */

.stTabs [data-baseweb="tab-list"] {
  gap: 8px;
  background-color: rgba(22, 27, 34, 0.6);
  backdrop-filter: blur(8px);
  padding: 6px;
  border-radius: 14px;
  border-bottom: none;
  box-shadow: inset 0 1px 3px rgba(0,0,0,0.2);
}

.stTabs [data-baseweb="tab"] {
  height: 42px; white-space: pre-wrap;
  background-color: transparent !important;
  border-radius: 10px !important;
  padding: 10px 24px; color: #8b949e;
  font-weight: 600; transition: all 0.2s;
  border: none; margin: 0;
}

.stTabs [data-baseweb="tab"]:hover {
  color: #c9d1d9;
  background-color: rgba(255,255,255,0.05) !important;
}

.stTabs [aria-selected="true"] {
  background-color: rgba(88, 166, 255, 0.15) !important;
  color: #58a6ff !important;
  border: 1px solid rgba(88, 166, 255, 0.3) !important;
  box-shadow: 0 4px 10px rgba(88, 166, 255, 0.1);
}

/* Sliders / Inputs aesthetic tweaks */
div[data-baseweb="slider"] { padding-top: 10px; }

```

```

/* Primary Button */
button[kind="primaryFormSubmit"], button[kind="primary"] {
    background: linear-gradient(90deg, #30e896 0%, #00c3ff 100%) !important;
    color: #1a1a1a !important;
    font-weight: 600 !important;
    border: none !important;
    border-radius: 12px !important;
    padding: 10px !important;
    transition: all 0.3s ease !important;
    box-shadow: 0 8px 30px rgba(48, 232, 150, 0.3) !important;
}

button[kind="primaryFormSubmit"]:hover, button[kind="primary"]:hover {
    background: linear-gradient(90deg, #30e896 0%, #00c3ff 100%) !important;
    box-shadow: 0 8px 30px rgba(48, 232, 150, 0.6) !important;
    transform: scale(1.02) !important;
    color: #1a1a1a !important;
    border: none !important;
}

div[data-testid="stForm"] { border: none !important; padding: 0 !important; }
</style>

"""", unsafe_allow_html=True)
else:

# Light Mode CSS (Highly Aesthetic)
st.markdown("""
<style>

/* Hide Streamlit Clutter */

#MainMenu {visibility: hidden;}

footer {visibility: hidden;}

header {visibility: hidden;}

```

```

/* App Background */

.stApp { background-color: #f6f8fa; color: #24292f; }

.css-1d391kg, .css-18ni7ap { background-color: #ffffff; border-right: 1px solid #d0d7de;}


/* Glassmorphic Metric Cards */

.metric-card {
    background: #ffffff;
    border: 1px solid rgba(0,0,0,0.05);
    border-radius: 12px;
    padding: 20px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.03);
    transition: all 0.3s cubic-bezier(0.25, 0.8, 0.25, 1);
    position: relative;
    overflow: hidden;
}

.metric-card:hover {
    transform: translateY(-4px);
    box-shadow: 0 8px 20px rgba(32, 201, 151, 0.1);
    border: 1px solid rgba(32, 201, 151, 0.3);
}

.metric-value { font-size: 38px; font-weight: bold; color: #24292f; text-shadow: 0 0 10px
rgba(0,0,0,0.05); }

.metric-title { font-size: 13px; color: #20c997; text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }

h1, h2, h3 { color: #0969da; }


/* Unique Feature Box */

.unique-feature {
    background: rgba(9, 105, 218, 0.03);
    backdrop-filter: blur(10px);

```

```
border: 1px solid rgba(9, 105, 218, 0.15);  
border-radius: 16px; padding: 25px; margin-top: 20px;  
box-shadow: inset 0 0 20px rgba(9,105,218,0.02);  
}
```

```
/* Modern Pill Tabs */
```

```
.stTabs [data-baseweb="tab-list"] {  
  gap: 8px;  
  background-color: rgba(208, 215, 222, 0.3);  
  backdrop-filter: blur(8px);  
  padding: 6px;  
  border-radius: 14px;  
  border-bottom: none;  
  box-shadow: inset 0 1px 3px rgba(0,0,0,0.05);  
}
```

```
.stTabs [data-baseweb="tab"] {  
  height: 42px; white-space: pre-wrap;  
  background-color: transparent !important;  
  border-radius: 10px !important;  
  padding: 10px 24px; color: #57606a;  
  font-weight: 600; transition: all 0.2s;  
  border: none; margin: 0;  
}
```

```
.stTabs [data-baseweb="tab"]:hover {  
  color: #24292f;  
  background-color: rgba(0,0,0,0.03) !important;  
}
```

```
.stTabs [aria-selected="true"] {  
  background-color: rgba(9, 105, 218, 0.1) !important;  
  color: #0969da !important;  
  border: 1px solid rgba(9, 105, 218, 0.2) !important;
```

```

        box-shadow: 0 4px 10px rgba(9, 105, 218, 0.05);
    }
    div[data-baseweb="slider"] { padding-top: 10px; }

    /* Primary Button */
    button[kind="primaryFormSubmit"], button[kind="primary"] {
        background: linear-gradient(90deg, #30e896 0%, #00c3ff 100%) !important;
        color: #1a1a1a !important;
        font-weight: 600 !important;
        border: none !important;
        border-radius: 12px !important;
        padding: 10px !important;
        transition: all 0.3s ease !important;
        box-shadow: 0 8px 30px rgba(48, 232, 150, 0.2) !important;
    }
    button[kind="primaryFormSubmit"]:hover, button[kind="primary"]:hover {
        background: linear-gradient(90deg, #30e896 0%, #00c3ff 100%) !important;
        box-shadow: 0 8px 30px rgba(48, 232, 150, 0.4) !important;
        transform: scale(1.02) !important;
        color: #1a1a1a !important;
        border: none !important;
    }
    div[data-testid="stForm"] { border: none !important; padding: 0 !important; }
</style>
"""', unsafe_allow_html=True)

```

--- Data Loading ---

@st.cache_data

def load_data():

try:

df = pd.read_csv("cluster_telemetry.csv")

```

df['timestamp'] = pd.to_datetime(df['timestamp'])

return df

except FileNotFoundError:

    return pd.DataFrame(columns=['timestamp', 'hour_of_day', 'day_of_week', 'avg_cpu_load',
'avg_memory_load', 'network_traffic_gbps', 'safe_shutdown_count'])

@st.cache_resource
def load_model():
    try:
        return joblib.load("rf_cluster_model.pkl")
    except FileNotFoundError:
        return None

df = load_data()
model = load_model()

# --- Sidebar ---

st.sidebar.image("https://cdn-icons-png.flaticon.com/512/3063/3063124.png", width=60)
st.sidebar.title("Eco-Scale Control")
st.sidebar.markdown("Predictive Cluster-Level Energy Optimization.")

with st.sidebar.expander("⚙️ Infrastructure Specs", expanded=True):
    total_servers = st.slider("Total Servers in Cluster", min_value=100, max_value=10000, value=1000,
step=100, help="The total hardware nodes running in your datacenter.")

    power_per_server = st.number_input("Avg Power per Server (Watts)", value=400, help="Average
power consumed by a single node at idle/moderate load.")

    electricity_cost = st.number_input("Electricity Cost (₹/kWh)", value=10.0, help="Cost of
commercial electricity. Example: ₹10/kWh.")

    carbon_intensity = st.number_input("Carbon Intensity (kg CO2/kWh)", value=0.45,
help="Emissions factor for your local energy grid. E.g., 0.45 for mixed grid.")

with st.sidebar.expander("📢 Notifications"):

```



```
enable_whatsapp = st.checkbox("Enable WhatsApp Notifications", help="Get automated alerts via WhatsApp API.")
```

```
phone_number = ""
```

```
if enable_whatsapp:
```

```
    phone_number = st.text_input("WhatsApp Number", value="", placeholder="+1234567890",  
help="Must include the country code without spaces (e.g., +91).")
```

```
st.sidebar.markdown("---")
```

```
st.sidebar.info("Model: Random Forest Regressor 🌲 ")
```

```
st.sidebar.info("Target: Dynamic Scale Extrapolation")
```

```
if df.empty or model is None:
```

```
    st.warning("Data or model not found. Please run `python build_project.py` to train the new  
Random Forest cluster model.")
```

```
    st.stop()
```

```
# Calculate Main Metrics for Live Tab
```

```
latest_data = df.iloc[-1] if not df.empty else None
```

```
current_predicted_shutdowns = int(latest_data['safe_shutdown_count'] * (total_servers / 1000)) if  
latest_data is not None else 0
```

```
power_saved_kw = (current_predicted_shutdowns * power_per_server) / 1000
```

```
daily_savings_kwh = power_saved_kw * 24
```

```
monthly_money_saved = daily_savings_kwh * 30 * electricity_cost
```

```
monthly_co2_saved_kg = daily_savings_kwh * 30 * carbon_intensity
```

```
# Initialize State for AI Simulator
```

```
if 'sim_results' not in st.session_state:
```

```
    st.session_state.sim_results = None
```

```
# --- Tabs Implementation (Re-ordered Simulator to Default) ---
```

```
tab_sim, tab_live, tab_help = st.tabs(["🧠 AI Simulator", "📡 Live Telemetry", "📘 Project Help"])
```

```
with tab_sim:
```

```
    st.markdown("<h2 style='color: #20c997; font-weight: 800;'>Eco-Scale Intelligence Center</h2>",  
unsafe_allow_html=True)
```

```
    st.markdown("<p style='color: #8b949e; margin-bottom: 30px;'>Real-time infrastructure telemetry  
and predictive capability optimization dashboard.</p>", unsafe_allow_html=True)
```

```
with st.form("simulation_form"):
```

```
    col1, col2, col3 = st.columns(3)
```

```
    with col1:
```

```
        sim_hour = st.slider("Hour of Day (0-23)", 0, 23, 14, help="Peak hours generally have less  
suspensible servers.")
```

```
        sim_day = st.slider("Day of Week (0-6)", 0, 6, 2, help="0=Monday, 6=Sunday. Weekends  
typically allow more scaling.")
```

```
    with col2:
```

```
        sim_cpu = st.slider("Avg CPU Load (%)", 0, 100, 35, help="Target processing volume.")
```

```
        sim_mem = st.slider("Avg Memory Load (%)", 0, 100, 45, help="Target memory volume.")
```

```
    with col3:
```

```
        sim_net = st.slider("Network Traffic (Gbps)", 0.0, 100.0, 18.5, help="High bandwidth spikes  
limit potential server shutdowns.")
```

```
    st.markdown("<br>", unsafe_allow_html=True)
```

```
    submit = st.form_submit_button("Initialize Forecasting Engine", type="primary",  
use_container_width=True)
```

```
if submit:
```

```
    with st.spinner("Analyzing high-dimensional cluster space..."):
```

```
        time.sleep(0.5)
```

```
    features = pd.DataFrame({
```

```
        'hour_of_day': [sim_hour],
```

```
        'day_of_week': [sim_day],
```

```
        'avg_cpu_load': [sim_cpu],
```

```

    'avg_memory_load': [sim_mem],
    'network_traffic_gbps': [sim_net]
})

```

```

base_pred = model.predict(features)[0]
pred_shutdowns = int(base_pred * (total_servers / 1000))
pred_shutdowns = max(0, min(total_servers, pred_shutdowns))
active_servers = total_servers - pred_shutdowns

```

```

sim_power_saved_kw = (pred_shutdowns * power_per_server) / 1000
sim_daily_savings_kwh = sim_power_saved_kw * 24
sim_monthly_money_saved = sim_daily_savings_kwh * 30 * electricity_cost
sim_monthly_co2_saved_kg = sim_daily_savings_kwh * 30 * carbon_intensity

```

```

st.session_state.sim_results = {
    'active': active_servers,
    'suspendable': pred_shutdowns,
    'savings': sim_monthly_money_saved,
    'carbon': sim_monthly_co2_saved_kg
}

```

```

if phone_number and pred_shutdowns > 0:

```

```

    try:

```

```

        whatsapp.send_prediction_alert(
            phone_number=phone_number,
            sim_cpu=sim_cpu,
            sim_mem=sim_mem,
            sim_net=sim_net,
            action_title=f"Shutdown {pred_shutdowns} Servers",
            action_desc=f"Leave {active_servers} servers running. Safe under current {sim_cpu}%

```

```

CPU load."

```

```

    )

    st.toast("WhatsApp alert triggered!")

except Exception as e:

    st.error(f"Failed to send WhatsApp alert: {e}")


if st.session_state.sim_results:

    res = st.session_state.sim_results


    st.markdown("<br>", unsafe_allow_html=True)

    sc1, sc2, sc3, sc4 = st.columns(4)

    with sc1:

        st.markdown(f'<div class="metric-card"><div class="metric-title">ACTIVE FLEET</div><div
class="metric-value">{res["active"]:.}</div></div>', unsafe_allow_html=True)

    with sc2:

        st.markdown(f'<div class="metric-card"><div class="metric-title">SAFELY SUSPENDED
(SIM)</div><div class="metric-value">{res["suspendable"]:.}</div></div>', unsafe_allow_html=True)

    with sc3:

        st.markdown(f'<div class="metric-card"><div class="metric-title">PROJECTED OPEX SAVINGS
(SIM)</div><div class="metric-value">₹{res["savings"]:.0f}</div></div>', unsafe_allow_html=True)

    with sc4:

        st.markdown(f'<div class="metric-card"><div class="metric-title">CARBON AVOIDANCE RATE
(SIM)</div><div class="metric-value">{res["carbon"]/1000:.1f} Tons</div></div>',
unsafe_allow_html=True)


with tab_live:

    # --- Top Metrics Row ---

    st.subheader("Current Cluster Snapshot")

    col1, col2, col3, col4 = st.columns(4)

    with col1:

        st.markdown(f'<div class="metric-card"><div class="metric-title">TOTAL CLUSTER SIZE</div><div
class="metric-value">{total_servers:.}</div></div>', unsafe_allow_html=True)

    with col2:

```

```
st.markdown(f'<div class="metric-card"><div class="metric-title">SAFELY IDLE  
SERVERS</div><div class="metric-value">{current_predicted_shutdowns:,}</div></div>',  
unsafe_allow_html=True)
```

with col3:

```
st.markdown(f'<div class="metric-card"><div class="metric-title">EST.  
SAVINGS/MONTH</div><div class="metric-value">₹{monthly_money_saved:,.0f}</div></div>',  
unsafe_allow_html=True)
```

with col4:

```
st.markdown(f'<div class="metric-card"><div class="metric-title">CO2  
AVOIDED/MONTH</div><div class="metric-value">{monthly_co2_saved_kg/1000:,.1f}  
Tons</div></div>', unsafe_allow_html=True)
```

```
st.markdown("<br>", unsafe_allow_html=True)
```

--- Charts ---

```
st.subheader("7-Day Cluster Telemetry & Optimization Potential")
```

Show last 168 hours (7 days)

```
plot_df = df.tail(168).copy()
```

```
plot_df['safe_shutdown_count'] = (plot_df['safe_shutdown_count'] * (total_servers /  
1000)).astype(int)
```

```
fig = go.Figure()
```

Add CPU Line

```
fig.add_trace(go.Scatter(  
    x=plot_df['timestamp'], y=plot_df['avg_cpu_load'],  
    mode='lines', name='Avg CPU Load (%)', line=dict(color='#0969da' if light_mode else '#58a6ff',  
width=3), yaxis='y1'  
))
```

Add Safe Shutdown Count as bars

```
fig.add_trace(go.Bar(  
    x=plot_df['timestamp'], y=plot_df['safe_shutdown_count'],
```

```

        name='Servers Powered Down', marker_color='#2ea043', opacity=0.5, yaxis='y2'
    ))

    bg_color = 'rgba(0,0,0,0)' # Transparent Plotly BG to match Glass UI
    font_color = '#24292f' if light_mode else '#c9d1d9'
    grid_color = '#d0d7de' if light_mode else 'rgba(255,255,255,0.05)'

    fig.update_layout(
        plot_bgcolor=bg_color, paper_bgcolor=bg_color, font_color=font_color,
        yaxis=dict(title='Cluster CPU Load (%)', range=[0, 100], gridcolor=grid_color),
        yaxis2=dict(title='Servers Shutdown Count', overlaying='y', side='right', range=[0, total_servers],
            showgrid=False),
        xaxis=dict(gridcolor=grid_color),
        hovermode="x unified", margin=dict(l=0, r=0, t=30, b=0),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )

    st.plotly_chart(fig, use_container_width=True)

```

with tab_help:

```

    st.markdown("""
    ##  Project Context & Help

    Welcome to Eco-Scale, an intelligent infrastructure scaling platform developed to reduce
    scope-2 carbon emissions by dynamically shutting down redundant data-center elements during
    periods of low activity.

    ###  GitHub Repository

    * **URL**: [Green-cloud-Infrastructure-for-
    sustainability](https://github.com/anshvk05/GreenCloud-Infrastructure-for-sustainability)

    ###  Core Features

    - Telemetry Analysis: Uses a Random Forest Regressor to predict the upper bounds of safe
    cluster suspension.
    """)

```

- **Capacity Safeguards**: Hard limits ensure cluster utilization never breaches thresholds, maintaining performance.
- **Cost & Carbon Extrapolation**: Real-time calculus of ROI and sustainability impact.
- **WhatsApp Automated Ops**: API-driven notification pipeline for immediate human-in-the-loop intervention.

📊 How Savings are Calculated

- **Power Consumption** = `Suspendable Servers × Power/Server`
- **Money Saved** = `Power Saved (kW) × 24h × 30d × Electricity Cost/kWh`
- **Carbon Avoided** = `Power Saved (kW) × 24h × 30d × Carbon Intensity (kg CO<sub>2</sub>/kWh)`

🛠 Troubleshooting & Model Info

If predictions skew, verify the model input schemas in `cluster\_telemetry.csv` and retrain using `build\_project.py`. The machine learning algorithm correlates temporal structures (time-of-day, day-of-week) against cluster pressure variables (CPU, Memory, Network) to interpolate secure suspension counts.

```
"""
```

```
st.markdown("---")
```

```
st.markdown(
```

```
"<div style='text-align: center; color: #8b949e; font-size: 12px; opacity: 0.6;'>"
```

```
"Algorithm: Random Forest Regression | Target: Dynamic Cluster Size | Built for Environmental Hackathon"
```

```
"</div>", unsafe_allow_html=True
```

```
)
```